

6.设备端NPU环境准备(含更新NPU驱动方法)

1. NPU驱动版本确认

查询命令：

代码块

```
1 dmesg | grep -i rknpu
2 或
3 cat /sys/kernel/debug/rknpu/version
4 或
5 cat /sys/kernel/debug/rknpu/driver_version
6 或
7 cat /proc/debug/rknpu/driver_version
```

查询结果：

代码块

```
1 RKNPU driver: vX.X.X
```

X.X.X表示版本号，例如0.9.2。

Rockchip的固件均自带RKNPU驱动。若以上命令均查询不到NPU驱动版本，则可能为第三方固件未安装RKNPU驱动，需要打开kernel config文件的“CONFIG_ROCKCHIP_RKNPU=y”选项，重新编译内核驱动并烧录。建议RKNPU驱动版本 $\geq 0.9.2$ 。

目前最新的是0.9.8，我建议直接就升级到最新的版本，具体升级的方法在最后可以看到。

```
root@topeet:~# cat /sys/kernel/debug/rknpu/version
RKNPU driver: v0.9.8
```

2. NPU连板环境确认

RKNN-Toolkit2的连板调试功能依赖板端的RKNN Server程序，该程序是一个运行在开发板上的后台代理服务，用于接收PC通过USB传输过来的命令和数据，然后调用相应的运行时接口，并返回相应结果给PC。所以在做连板调试前需要确认开发板是否已启动RKNN Server程序。

注：ARM64版本不需要通过连板调试，可跳过

检查方法：通过ADB工具进入开发板终端，查询是否有RKNN Server进程。

代码块

```
1 adb shell
2 ps | grep rknn_server
```

查询结果：

代码块

```
1 702 root 1192 S grep rknn_server
```

若查询到RKNN Server进程ID，则说明RKNN Server启动正常。若没有查询到RKNN Server进程ID，则执行以下命令手动启动RKNN Server后再进行查询。

RK356X/ RK3576 / RK3588 / RV1126B 的Linux系统手动启动RKNN Server：

代码块

```
1 nohup /usr/bin/rknn_server >/dev/null&
```

3. 更新 npu 连板推理运行库

RKNN-Toolkit2 的连板调试功能要求板端已安装 RKNPU2 环境，并且启动 RKNN Server 服务。以下是 RKNPU2 环境中的2个基本概念：

1. **RKNN Server**：一个运行在开发板上的后台代理服务，用于接收PC通过 USB 传输过来的数据，然后执行板端 Runtime 对应的接口，并返回结果给 PC。
2. **RKNPU2 Runtime 库**
 - librknnrt.so: 是用于 RK3562 / RK3566 / RK3568 / RK3576 / **RK3588** / RV1126B 板端的 Runtime 库。
 - librknnmrt.so: 是用于 RV1103 / RV1103B / RV1106 / RV1106B板端的 Runtime 库。

如果板端没有安装 RKNN Server，Runtime 库不存在，或者 RKNN Server 和 Runtime 库的版本不一致，都需要重新安装或更新。

注意：

1. 若使用动态维度输入的 RKNN 模型，则要求 RKNN Server 和 RKNPU2 Runtime 库版本 $\geq 1.5.0$ 。
2. 若使用大于2GB的模型，要求rknn_server版本 $\geq 2.0.0b0$ 。
3. 在RV1103 / RV1103B / RV1106 / RV1106B等小内存平台上，建议rknn_server版本 $\geq 2.1.0$ 。

4. 要保证 RKNN Server、Runtime 库版本、RKNN-Toolkit2 版本一致，建议都安装最新的版本。
5. RK2118中的RKNN Server并不是用来做RKNN-Toolkit2的连板调试功能，而是用于DSP与MCU进行交互的服务。
6. ARM64版本无需依赖RKNN Server，所以不用启动和更新，但是调试功能依赖板端的RKNPU2 Runtime库，确保板端有这个库并更新到最新版

不管之前的版本是什么，如果没有更新过，就直接按照下面的方法来更新

查询RKNN Server服务和 librknnrt.so 库版本.

在这里如果大家是购买的某些厂商

代码块

```
1  # 查询rknn_server版本
2  strings /usr/bin/rknn_server | grep -i "rknn_server version"
3  # 显示rknn_server版本为X.X.X
4  # rknn_server version: X.X.X
5  # 查询librknnrt.so库版本
6  strings /usr/lib/librknnrt.so | grep -i "librknnrt version"
7  # 显示librknnrt库版本为X.X.X
8  # librknnrt version: X.X.X
```

更新RKNN Server服务和librknnrt.so库。

代码块

```
1  # 方法一
2  adb push runtime/Linux/rknn_server/aarch64/usr/bin/* /usr/bin
3  adb push runtime/Linux/librknn_api/aarch64/librknnrt.so /usr/lib
4
5  # 方法二
6  scp -r runtime/Linux/rknn_server/aarch64/usr/bin/*
   root@192.168.137.142:/usr/bin
7  scp -r runtime/Linux/librknn_api/aarch64/librknnrt.so
   root@192.168.137.142:/usr/lib
```

重启RKNN Server服务：

代码块

```
1  cd /usr/bin
2  chmod +x /usr/bin/rknn_server
3  chmod +x /usr/bin/start_rknn.sh
4  chmod +x /usr/bin/restart_rknn.sh
```

最终的版本如下：

```
root@topeet:/usr/bin# strings /usr/bin/rknn_server | grep -i "rknn_server version"
rknn_server version: 2.3.2 (1842325 build@2025-03-30T09:54:34)
root@topeet:/usr/bin# strings /usr/lib/librknnrt.so | grep -i "librknnrt version"
librknnrt version: 2.3.2 (429f97ae6b@2025-04-09T09:09:27)
root@topeet:/usr/bin# ps | grep rknn_server
 3750 pts/1    00:00:00 rknn_server
root@topeet:/usr/bin# cat /sys/kernel/debug/rknpu/version
RKNPU driver: v0.9.8
root@topeet:/usr/bin#
```

4. 查看RKNN Server详细日志

进入开发板终端，设置日志等级。

代码块

```
1  adb shell
2  export RKNN_SERVER_LOGLEVEL=5
```

重启RKNN Server服务可查看详细日志。

针对RK356X / RK3576 / RK3588 / RV1126B 平台，执行下列命令。

代码块

```
1  nohup /usr/bin/rknn_server >/dev/null&
```

补充：更新RKNPU

注意：退出这个conda环境在编译，博主试了很多次，没退出编译不成功。

注意：退出这个conda环境在编译，博主试了很多次，没退出编译不成功。

注意：退出这个conda环境在编译，博主试了很多次，没退出编译不成功。

如果大家使用的是讯为的开发板可以直接使用我编译好的镜像。

1.将编译源码需要使用的两个包传送到Ubuntu上面。

```
(base) topeet@ubuntu:~$ ls
Desktop      Pictures      snap
Documents    Public        Templates
Downloads    rk3588-linux_20260427.tar.xz  Videos
miniconda3   rknn
Music        rknpu_driver_0.9.8_20241009.tar.bz2
(base) topeet@ubuntu:~$
```

2.解压该压缩包，将其中的rknpu 驱动代码覆盖到当前内核代码目录(kernel/drivers/rknpu)。

3.编译内核， ./build.sh ubuntu20_update 。

```
(base) topeet@ubuntu:~$ cd rk3588-linux/
(base) topeet@ubuntu:~/rk3588-linux$ ./build.sh ubuntu20_update
这是第一次运行脚本，请输入您的用户密码。
当前运行的系统为 focal.
正在检查网络连接...
网络连接正常，开始安装依赖包。
正在安装 Ubuntu 20.04 / 22.04 编译所需依赖包...
[sudo] password for topeet:
```

如编译失败：

安装依赖

代码块

```
1 sudo apt-get install -y uuid uuid-dev zlib1g-dev liblz-dev liblzo2-2 liblzo2-
dev lzop git curl u-boot-tools mtd-utils openjdk-8-jdk device-tree-compiler
gdisk m4 gnupg flex bison gperf libsdl1.2-dev libesd-java squashfs-tools build-
essential zip libncurses5-dev pngcrush schedtool libxml2 libxml2-utils
xsltproc libc6-dev g++-multilib lib32z1-dev lib32ncurses-dev lib32readline-dev
gcc-multilib libswitch-perl libssl-dev unzip liblz4-tool ssh make vim expect
g++ patchelf chrpath gawk texinfo diffstat binfmt-support qemu-user-static
live-build fakeroot cmake python3-pip rsync subversion python-protobuf sed
binutils wget python-is-python2 libncurses5 bzip2 cvs mercurial patch gzip bzip2
perl tar cpio file bc android-sdk-libsparse-utils android-sdk-ext4-utils
```

打开 kernel/drivers/rknpu/rknpu_devfreq.c，把第237行注释掉。

```
235 static const struct rockchip_opp_data rk3576_npu_opp_data = {
236     .set_read_margin = rk3576_npu_set_read_margin,
237     // .set_soc_info = rockchip_opp_set_low_length,
238 #if KERNEL_VERSION(6, 1, 0) <= LINUX_VERSION_CODE
239     .config_regulators = npu_opp_config_regulators,
240     .config_clks = npu_opp_config_clks,
241 #endif
242 };
243
244 static const struct rockchip_opp_data rk3588_npu_opp_data = {
```

打开 kernel/include/linux/mm.h ，在文件合适的位置（比如开头或结尾）添加以下两个函数的定义：

```
static inline void vm_flags_set(struct vm_area_struct *vma, vm_flags_t flags)
{
    vma->vm_flags |= flags;
}
static inline void vm_flags_clear(struct vm_area_struct *vma, vm_flags_t flags)
{
    vma->vm_flags &= ~flags;
}
```

编译成功即可看到下图的结果。

```
address=0x00006000
Add file: ./boot.img
boot,Add file: ./boot.img done,offset=0x487000,size=0x266c200,userspace=0x4cd9,f
lash_address=0x00008000
Add file: ./recovery.img
recovery,Add file: ./recovery.img done,offset=0x2af3800,size=0x6387000,userspace
=0xc70e,flash_address=0x00028000
Add file: ./rootfs.img
rootfs,Add file: ./rootfs.img done,offset=0x8e7a800,size=0xedc00000,userspace=0x
1db800,flash_address=0x00078000
Add CRC...
Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.23*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Running mk-updateimg.sh - build_updateimg succeeded.
Running 99-all.sh - build_ubuntu20_all succeeded.
topeet@ubuntu:~/rk3588-linux$
```

4.将新编译的内核烧录到设备中。

